DataCopilot – Capstone Project Proposal

Oct 30, 2026 · @Mritunjay Sharma

Executive Summary

DataCopilot is a web application that lets anyone ask questions about a CSV/Excel file or a SQL database in plain English or Hinglish and get a verified answer, a chart, and the exact SQL used.

Unlike existing AI data tools, DataCopilot focuses on reliability. A large language model (LLM) writes SQL, but the system validates every query against the data schema and blocks unsafe operations before execution. A machine-learning classifier also detects questions the data cannot answer, so the system declines instead of guessing.

The project will be evaluated on the BIRD text-to-SQL benchmark plus custom CSV, Hinglish, and unanswerable question sets. The main result will measure how much validation and answerability checking reduce incorrect answers compared with a baseline LLM pipeline. The application will be deployed on AWS or GCP using Docker, and the experimental results will form the basis of an independent research paper.

Background and Context

Organizations of every size store important information in spreadsheets and databases, but extracting insights from them still requires SQL, Python, or statistics skills. Most business users, small-business owners, and students do not have these skills and depend on analysts for even simple questions.

Large language models have made natural-language data analysis possible. Tools such as ChatGPT's data analysis feature, Julius AI, PandasAI, and Vanna let users ask questions about data in plain language. Enterprise products such as Power BI Copilot and Databricks Genie offer similar features at a high cost and with significant setup.

However, LLMs can generate incorrect SQL or code that runs successfully but returns wrong numbers. They may also answer questions the data cannot support, producing confident but unsupported results. Text-to-SQL research benchmarks such as BIRD show that even strong models make frequent errors on realistic, messy databases. For decisions based on data, an answer that looks correct but is wrong can be worse than no answer.

Problem Statement

Non-technical users cannot easily analyze their own data, and current AI tools that promise to help often execute LLM-generated queries without verifying them. This leads to three specific problems:

1. **Incorrect answers:** queries reference wrong columns, apply wrong filters, or aggregate incorrectly, yet still return a result.
2. **Unsupported answers:** the system answers questions that the data does not contain, instead of saying so.
3. **Unsafe execution:** generated queries could modify or delete data if not restricted.

Why it matters: decisions about sales, budgets, or operations based on wrong numbers can cost money and trust. Users also cannot check an answer if the tool does not show how it was produced.

Stakeholders affected:

- Small-business owners and managers who keep data in Excel or simple databases
- Students and researchers working with datasets
- Data analysts who spend time answering repetitive questions
- Organizations that need trustworthy, auditable answers from their data

Opportunity: a system that checks each query before running it, refuses unanswerable questions, and shows its work can make natural-language data analysis both accessible and reliable.

Business Justification

DataCopilot reduces the time and skill needed to get trustworthy answers from data, while lowering the risk of acting on wrong numbers.

Existing option	Limitation	DataCopilot's approach
ChatGPT data analysis, Julius AI	Runs generated code with limited validation; paid tiers	Validates every SQL query before execution
PandasAI, Vanna (open source)	Focus on generation, not measured reliability	Measures and reports error reduction on a benchmark
Power BI Copilot, Databricks Genie	Expensive, enterprise setup required	Free, open-source, works on a single CSV
Manual analysis by an analyst	Slow, requires SQL/Python skills	Instant answers in English or Hinglish

Value delivered:

- **Accessibility:** non-technical users, including Hinglish speakers, can query data directly.
- **Trust:** every answer shows the SQL used and a verification status.
- **Safety:** read-only enforcement prevents accidental data changes.
- **Cost:** built entirely on free and open-source tools, making it suitable for small businesses and students.

Project Objectives

1. Build a system that accepts CSV/Excel uploads and connections to SQLite and PostgreSQL databases.
2. Automatically profile each dataset and generate insights (summary statistics, correlations, missing values, anomalies).
3. Train and compare ML classifiers that predict whether a question is answerable from the given data.
4. Use an LLM to translate English and Hinglish questions into SQL.
5. Validate generated SQL for schema correctness and read-only safety before execution.
6. Return answers as automatically selected charts, plain-language explanations, and the SQL used.
7. Evaluate the system on the BIRD benchmark and custom question sets, comparing a baseline pipeline with the validated pipeline.
8. Deploy the application on AWS or GCP using Docker, with user authentication and logging.

9. Document the full system in a professional GitHub repository and write an independent research paper from the results.

Expected Outcomes

- A working, cloud-deployed DataCopilot web application accessible through a public link.
- A trained answerability classifier with documented model comparison results.
- A benchmark evaluation showing accuracy, wrong-answer rate, and correct refusals for the baseline and validated pipelines, in English and Hinglish.
- A reproducible evaluation dataset of custom CSV, Hinglish, and unanswerable questions.
- A complete GitHub repository with source code, documentation, architecture diagrams, and test reports.
- All capstone deliverables: reports for each module, user guide, final report, presentation, and live demo.
- A draft independent research paper based on the experimental findings.

Success Criteria

The project succeeds if the validated pipeline measurably reduces wrong answers and the deployed system meets the targets below.

Area	Measure	Target
Reliability	Wrong-answer rate, validated vs baseline pipeline	Statistically significant reduction (McNemar's test, $p < 0.05$)
Answerability classifier	F1-score on held-out questions	≥ 0.85
Safety	Unsafe queries (INSERT, UPDATE, DELETE, DROP) blocked	100% of test cases
Hinglish	Execution accuracy gap vs English	Measured and reported
Performance	Median response time on cloud deployment	$\leq$ 10 seconds
Deployment	Application accessible via public URL with login	Achieved

Area	Measure	Target
Usability	User acceptance testing with 5+ users	Average rating ⩾ 4 out of 5
Documentation	All module deliverables in GitHub	Complete and on time

Project Scope

Included:

- Data sources: CSV and Excel uploads (loaded into DuckDB), SQLite and PostgreSQL connections
- Questions in English and Hinglish
- Automatic dataset profiling and insights
- Answerability classification
- LLM-based SQL generation with schema and read-only validation
- Automatic chart selection (bar, line, pie, scatter, single value) using Plotly
- Transparency panel showing the generated SQL and verification status
- Evaluation on the BIRD development set and custom question sets
- Cloud deployment with Docker, user login, and logging

Excluded:

- Other databases (MySQL, MongoDB, data warehouses)
- Real-time or streaming data
- Editing, inserting, or deleting data
- Very large files (over approximately 100 MB per upload)
- Languages other than English and Hinglish
- Training or fine-tuning a custom LLM

Assumptions and Constraints

Assumptions:

- Uploaded data is tabular, with a header row and reasonably clean column names.
- Open-source LLMs (7–8B parameters, run through Ollama) are capable enough to generate useful SQL for evaluation.
- The BIRD development set remains publicly available for research use.

- Users ask analytical questions (counts, totals, trends, comparisons), not open-ended predictions.

Constraints:

- **Time:** eight weeks, with one module's deliverables due each week.
- **Budget:** no paid credits; the project uses free and open-source tools and cloud free tiers.
- **Hardware:** local experiments run on a laptop GPU with 8 GB of VRAM, which limits model size.
- **Cloud resources:** free-tier servers (about 1 GB RAM) cannot host an LLM, so the deployed demo will call an LLM API or a lightweight model.
- **Team:** a single developer handles all roles, from design to deployment.

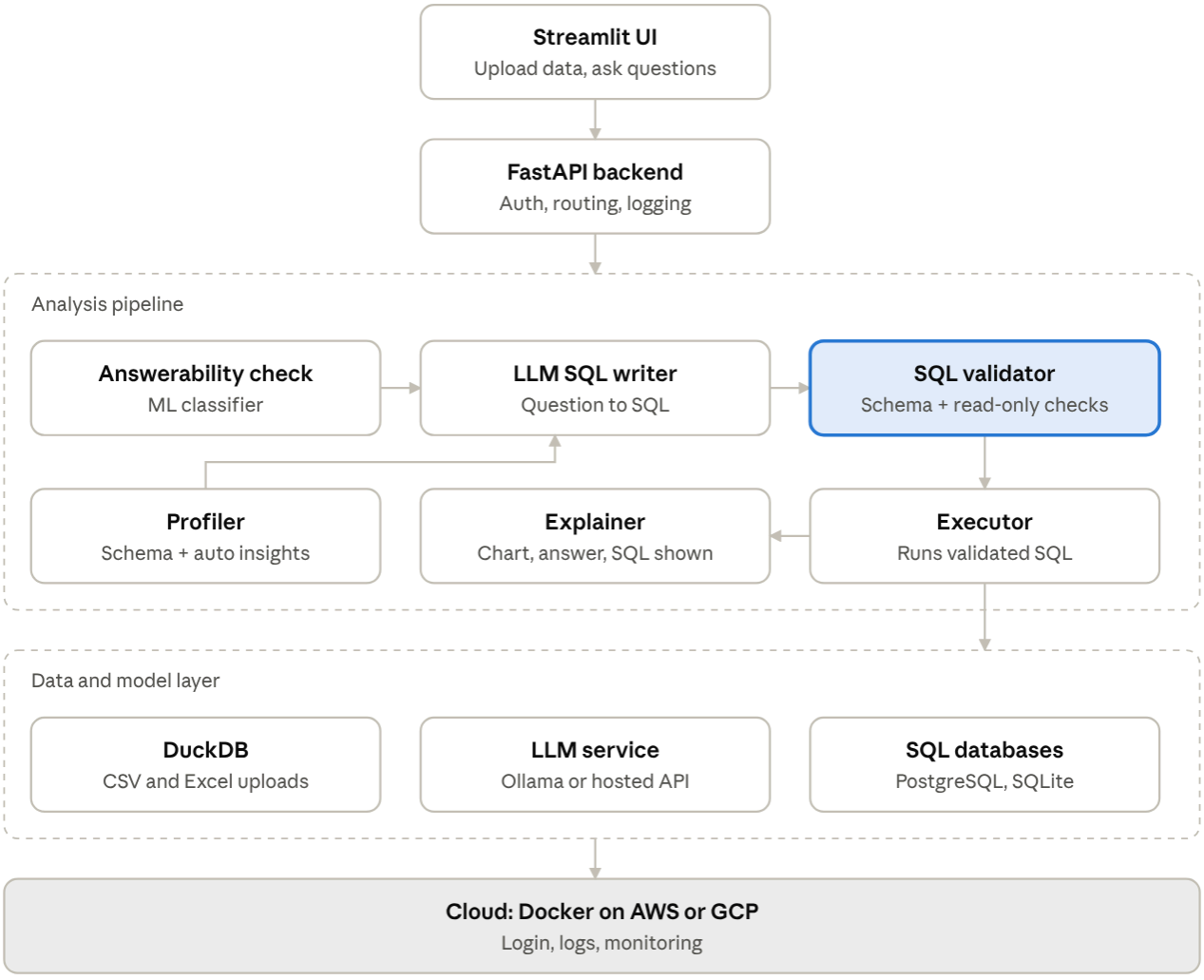
Preliminary Technology Stack

Layer	Tools	Purpose
Programming	Python	Core language
Data processing	Pandas, NumPy, DuckDB	Profiling, insights, CSV/Excel querying
Databases	SQLite, PostgreSQL, SQLAlchemy	External data sources
Machine learning	scikit-learn, XGBoost	Answerability classifier
LLM	Ollama (Qwen2.5-Coder, Llama 3.1) or hosted API	SQL generation
Validation	sqlglot	SQL parsing, schema and safety checks
Backend	FastAPI	API and pipeline orchestration
Frontend	Streamlit	User interface
Visualization	Plotly	Interactive charts
Testing	Pytest, BIRD evaluation scripts	Unit tests and benchmark evaluation
Deployment	Docker, AWS or GCP	Cloud hosting
Version control	Git, GitHub	Code and artefact management

The final choice of LLM and cloud provider will be confirmed in Module 2 based on accuracy, cost, and free-tier limits.

High-Level Solution Architecture

The LLM only writes SQL; the validator and database decide the answer



DataCopilot architecture · 4 layers

Every question becomes SQL: CSV and Excel files are loaded into DuckDB, so one validated pipeline serves both files and databases. The LLM never calculates; it only writes SQL, which the validator checks before the database computes the answer.

References

1. Li, J., et al. (2023). *Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs (BIRD).* NeurIPS 2023. bird-bench.github.io

2. Yu, T., et al. (2018). *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task.* EMNLP 2018.
3. McNemar, Q. (1947). *Note on the sampling error of the difference between correlated proportions or percentages.* Psychometrika, 12(2), 153–157.
4. DuckDB documentation. duckdb.org
5. sqlglot: SQL parser and transpiler. github.com/tobymao/sqlglot
6. Ollama: run open-source LLMs locally. ollama.com
7. Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System.* KDD 2016.

References will be expanded with a full literature review in Module 2.